Lecture 28.10: Pre-Pruning Implementation in Python

📌 Topic Overview

In this lecture, we implement **Pre-Pruning** on the Titanic dataset. We systematically iterate over different tree depths (`max_depth`), discover the optimal depth where test accuracy peaks, and automate hyperparameter tuning with `GridSearchCV` .

🔍 The Systematic Depth Search Loop

In the video, the instructor writes a clean loop over candidate depths:

```
def find_best_max_depth(X_train, y_train, X_test, y_test, depth_range=range(2, 11)
    """
    Evaluates DecisionTreeClassifier test accuracy across max_depth values from 2
    to identify the optimal tree depth.
    """
    best_depth = None
    best_score = 0.0
    history = []

    for depth in depth_range:
        clf = DecisionTreeClassifier(max_depth=depth, random_state=42)
        clf.fit(X_train, y_train)
        acc = clf.score(X_test, y_test)

        history.append((depth, acc))
        if acc > best_score:
            best_score = acc
            best_depth = depth

    return best_depth, best_score, history
```

Actual Output from the Lecture:

- `for depth=2, accuracy = 0.772`
- `for depth=3, accuracy = 0.806`
- **`for depth=4, accuracy = 0.825`** ← **PEAK PERFORMANCE!**
- `for depth=5, accuracy = 0.795`
- `for depth=6, accuracy = 0.784`

- `for depth=7, accuracy = 0.780`
- `for depth=8, accuracy = 0.791`
- `for depth=9, accuracy = 0.765`
- `for depth=10, accuracy = 0.791`

Key Takeaway from the Results:

As `max_depth` increases from 2 to 4, the model learns meaningful feature splits and accuracy climbs to **82.5%**. Beyond depth 4, the tree starts memorizing noise, and testing accuracy steadily degrades! Thus, **`max_depth=4`** is the optimal sweet spot.

⚙️ Automated Tuning with `GridSearchCV`

Instead of tuning one parameter by hand, we can use Scikit-Learn's `GridSearchCV` to cross-validate multiple hyperparameters simultaneously (`max_depth`, `min_samples_leaf`, and `min_samples_split`).

```
from sklearn.model_selection import GridSearchCV
from sklearn.tree import DecisionTreeClassifier

def tune_tree_with_gridsearch(X_train, y_train):
    """
    Performs 5-fold GridSearchCV across max_depth and min_samples_leaf
    to identify optimal regularized hyperparameters.
    """
    param_grid = {
        'max_depth': [2, 3, 4, 5, 6],
        'min_samples_leaf': [2, 5, 10, 15]
    }
    grid = GridSearchCV(
        DecisionTreeClassifier(random_state=42),
        param_grid=param_grid,
        cv=5,
        scoring='accuracy'
    )
    grid.fit(X_train, y_train)
    return grid.best_estimator_, grid.best_params_, grid.best_score_
```

🔑 Key Takeaways

1. Pre-pruning via `max_depth` stops the tree from growing overly complex branches.
2. In the Titanic dataset, `depth=4` achieved the peak test accuracy of 82.5% (compared to 77.5% unpruned).

3. `GridSearchCV` automates the search across combinations of `max_depth`, `min_samples_leaf`, and `min_samples_split`.